



E-COMMERCE PLATFORM

Backend API Documentation

Frontend Integration Edition — endpoints ordered by consumption priority

Java 17 · Spring Boot 3 · Spring Security 6 · MySQL 8 · Flyway · Lombok

TOTAL APIS	DB TABLES	BASE URL	VERSION
35+	13	localhost:8080	v1.0

1. Project Overview

Purpose, architecture and how this document is organised

A full-featured multi-role e-commerce backend built with Spring Boot 3. It supports four user roles — Customer, Seller, Employee and Admin — and handles the complete e-commerce lifecycle: product catalogue management, cart & checkout, order fulfilment, payment processing, and seller payout settlement.

Request Flow

Client → Security Filter → REST Controller → Service Layer → Repository (JPA) → MySQL Database

Document Ordering — Frontend-First

This documentation has been reorganised (from the original 16-section reference) so that the APIs a frontend application actually calls come first. Sections 2–8 cover every endpoint a Customer- or Seller-facing web/mobile app will integrate with — Auth, Categories, Products, Cart, Orders, Payments and Seller Payouts/Analytics. Sections 9–10 cover Admin-only finance tooling. Sections 11–13 are backend reference material (error codes, database schema, entity model) kept at the end since the frontend does not call them directly.

2. Frontend Integration Map

Which screen calls which API — read this first

FRONTEND SCREEN / ACTION	API(S) CALLED
Login / Signup screen	POST /api/v1/auth/register, /login, /send-otp, /verify-otp, /google-login
Home / Category nav	GET /api/categories
Product listing & PDP	GET /api/v1/products, /products/{id}, /products/slug/{slug}, /products/category/{id}
Seller "Add Product" form	POST /api/v1/products · PUT/DELETE /api/v1/products/{id}
Cart / mini-cart widget	GET/POST/PUT/DELETE /api/v1/cart/user/{userId}/...
Checkout button	POST /api/v1/orders
Order history / tracking	GET /api/v1/orders/{id}, /orders/user/{userId} · PUT /orders/{id}/cancel
Payment confirmation	POST /api/v1/payments/process · GET /payments/order/{orderId}
Seller dashboard	GET /api/v1/analytics/seller/{id}/summary, /payouts/seller/{id}
Admin console (lower priority for storefront FE)	Finance, revenue-log & global payout endpoints – Section 9

3. Authentication APIs

Base path: `/api/v1/auth` — called first by every frontend app

POST `/api/v1/auth/register` — Register new user

Creates a new user. Defaults to `ROLE_CUSTOMER` if no role is specified.

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "secret123",
  "role": "ROLE_CUSTOMER", // optional
  "phoneNumber": "9876543210" // optional
}
```

Response — 201 Created

```
{ "userId": 1, "name": "John Doe", "email": "john@example.com", "role": "ROLE_CUSTOMER",
  "message": "User registered successfully!" }
```

FIELD	REQUIRED	NOTES
name	Required	Not blank. Split on first space into firstName + lastName
email	Required	Valid email format, unique in DB
password	Required	Minimum 6 characters
role	Optional	Defaults to <code>ROLE_CUSTOMER</code>
phoneNumber	Optional	Unique in DB

POST `/api/v1/auth/login` — Password login (Admin / Employee)

```
{ "email": "admin@ecom.com", "password": "admin123" }
```

Response — 200 OK

```
{ "userId": 1, "name": "Super Admin", "email": "admin@ecom.com", "role": "ROLE_ADMIN",
  "message": "User logged in successfully!" }
```

POST `/api/v1/auth/send-otp` — Send email OTP for Customer / Seller

Generates a 6-digit OTP, stored in the `otp_tokens` table.

```
{ "email": "customer@gmail.com" }
```

Response — 200 OK → "OTP sent successfully to customer@gmail.com"

POST `/api/v1/auth/verify-otp` — Verify OTP and login / auto-register

If the user does not already exist, this auto-creates a `ROLE_CUSTOMER` account.

```
{ "email": "customer@gmail.com", "otp": "482917" }
```

POST `/api/v1/auth/google-login` — Google OAuth login / auto-register

Accepts Google OAuth details. Auto-creates an account if the email is not found. Supports `ROLE_CUSTOMER` and `ROLE_SELLER`.

```
{
```

```
"email": "seller@gmail.com",  
"googleId": "google_uid_12345",  
"name": "Rahul Sharma",  
"profilePic": "https://lh3.google.com/photo.jpg",  
"role": "ROLE_SELLER"  
}
```

4. Category APIs

Base path: `/api/categories` — powers navigation & filters

GET `/api/categories` — *Get all categories (public)*

Returns the list of all categories. Public endpoint — no auth required. Typically called on app load to populate navigation.

GET `/api/categories/{id}` — *Get category by ID (public)*

Returns a single category. 404 Not Found if the ID doesn't exist.

POST `/api/categories` — *Create category (Admin)*

```
{ "name": "Electronics", "description": "All electronic products" }
```

Slug is auto-generated: "Electronics" becomes "electronics". Returns 201 Created.

PUT `/api/categories/{id}` — *Update category (Admin)*

```
{ "name": "Electronics & Gadgets", "description": "Updated description" }
```

DELETE `/api/categories/{id}` — *Delete category (Admin)*

Returns: "Category deleted successfully with id: {id}"

5. Product APIs

Base path: `/api/v1/products` — catalogue browsing & seller management

GET `/api/v1/products` — *Get all products (public)*

Returns the list of all active products — the primary listing-page call.

GET `/api/v1/products/{id}` — *Product by ID (public)*

404 if not found.

GET `/api/v1/products/slug/{slug}` — *Product by SEO slug (public)*

Useful for SEO-friendly frontend URLs, e.g. `/api/v1/products/slug/iphone-15-pro`

GET `/api/v1/products/category/{categoryId}` — *Products by category (public)*

Returns products filtered by category ID — used for category landing pages.

GET `/api/v1/products/seller/{sellerId}` — *Products by seller (public)*

Returns all products listed by a specific seller — used for seller storefronts.

POST `/api/v1/products` — *Create product (Seller/Admin)*

```
{
  "name": "iPhone 15 Pro",           // required
  "description": "Latest Apple...", // optional
  "price": 89999.00,                // required, >= 0
  "stockQuantity": 50,              // required, >= 0
  "imageUrl": "https://...",       // optional
  "categoryId": 1,                  // required
  "sellerId": 3                     // required
}
```

Response — 201 Created

```
{ "id": 10, "name": "iPhone 15 Pro", "slug": "iphone-15-pro", "price": 89999.00,
  "stockQuantity": 50, "categoryName": "Electronics", "isActive": true }
```

Slug is auto-generated from name; a numeric suffix is appended if the slug already exists.

PUT `/api/v1/products/{id}` — *Update product (Seller)*

Same request body as create. All fields can be updated.

DELETE `/api/v1/products/{id}` — *Delete product (Seller)*

Returns: "Product deleted successfully with id: {id}"

6. Cart APIs

Base path: `/api/v1/cart` — drives the cart / mini-cart widget

Each user has exactly ONE cart (1:1 relationship). The cart is auto-created on the first item add, and is automatically cleared after order placement.

GET `/api/v1/cart/user/{userId}` — *Get user cart*

```
{
  "cartId": 1, "userId": 5,
  "items": [{
    "cartItemId": 3, "productId": 10, "productName": "iPhone 15 Pro",
    "quantity": 2, "unitPrice": 89999.00, "subTotal": 179998.00
  }],
  "totalAmount": 179998.00
}
```

POST `/api/v1/cart/user/{userId}/items` — *Add item to cart*

```
{ "productId": 10, "quantity": 2 }
```

If the product is already in the cart, quantity is incremented. Returns the updated full cart.

PUT `/api/v1/cart/user/{userId}/items/{cartItemId}?quantity=3` — *Update item quantity*

Pass the new quantity as a query param. Returns the updated cart.

DELETE `/api/v1/cart/user/{userId}/items/{cartItemId}` — *Remove item from cart*

Removes the item and recalculates the total. Returns the updated cart.

DELETE `/api/v1/cart/user/{userId}/clear` — *Clear entire cart*

Removes all items and resets total to 0. Returns: "Cart cleared successfully for user id: {userId}"

7. Order APIs

Base path: `/api/v1/orders` — checkout & order tracking

POST `/api/v1/orders` — Place order from cart

```
{
  "userId": 5,                // required
  "shippingAddress": "123 MG Road, Pune", // required
  "paymentMethod": "COD"      // required: COD | RAZORPAY | PAYPAL
}
```

Response — 200 OK

```
{
  "id": 1, "orderTrackingNumber": "A1B2C3D4E5F6",
  "items": [{ "productId": 10, "productName": "iPhone 15 Pro", "quantity": 2, "subTotal":
179998.00 }],
  "totalAmount": 179998.00, "status": "PENDING", "paymentStatus": "PENDING"
}
```

Side effects on order placement: (1) stock is deducted for each product in the cart, and (2) the cart is automatically cleared after a successful order.

GET `/api/v1/orders/{id}` — Get order by ID

Returns the full order with all items. 404 if not found.

GET `/api/v1/orders/user/{userId}` — User order history

Returns the list of all orders placed by a specific user — the "My Orders" screen.

PUT `/api/v1/orders/{id}/cancel` — Cancel order

Sets status to CANCELLED. Stock is automatically restored for all items in the order.

GET `/api/v1/orders` — All orders (Admin view)

Returns all orders in the system.

PUT `/api/v1/orders/{id}/status?status=SHIPPED` — Update order status (Admin/Employee)

Pass status as a query param. Allowed values: PENDING · PROCESSING · SHIPPED · DELIVERED · CANCELLED

8. Payment APIs

Base path: `/api/v1/payments` — completes the checkout flow

POST `/api/v1/payments/process` — Process payment for order

```
{ "orderId": 1, "amount": 179998.00, "paymentMethod": "COD" }
```

Response — 200 OK

```
{
  "id": 1, "orderId": 1, "transactionId": "TXN_4A8B9C2D",
  "amount": 179998.00, "paymentMethod": "COD",
  "paymentStatus": "SUCCESS", "paymentDate": "2026-08-02T12:05:00"
}
```

Transaction ID is auto-generated as TXN_ + 8 random uppercase characters. The order's paymentStatus is automatically set to "PAID".

GET `/api/v1/payments/order/{orderId}` — Payment by order ID

Returns the payment record linked to the given order.

POST `/api/v1/payments/verify?transactionId=TXN_4A8B9C2D&isSuccess=true` — Verify payment status (Admin/gateway webhook)

Used to manually confirm or fail a payment (e.g. after a gateway webhook). isSuccess=true sets SUCCESS, false sets FAILED.

POST `/api/v1/payments/refund/{orderId}` — Refund payment (Admin)

Sets the payment status to REFUNDED and the order's paymentStatus to "REFUNDED".

9. Seller Payout & Analytics APIs

Base paths: `/api/v1/payouts`, `/api/v1/analytics` — powers the seller dashboard

GET `/api/v1/payouts/seller/{sellerId}` — All payouts for a seller

Returns all payout records (PENDING, PROCESSED, REJECTED) for the seller.

GET `/api/v1/analytics/seller/{sellerId}/summary` — Seller dashboard summary

```
{
  "totalProducts": 25,      // Products listed by this seller
  "totalOrders": 142,      // Orders from this seller's products
  "totalRevenue": 950000.00, // Revenue from PAID orders
  "pendingOrders": 8,
  "completedOrders": 120
}
```

GET `/api/v1/payouts/pending` — List all pending seller payouts (Admin)

Payout Flow: payouts are pre-calculated entries in the payouts table. Admin uses these APIs to view pending payouts and mark them processed after a bank transfer, along with a UTR / transaction reference number.

```
[{
  "payoutId": 1, "sellerId": 3, "storeName": "TechHub Store",
  "accountNumber": "9876543210", "ifscCode": "HDFC0001234", "bankName": "HDFC Bank",
  "netPayableAmount": 45200.00, "status": "PENDING"
}]
```

POST `/api/v1/payouts/process` — Process payout — mark as completed (Admin)

```
{ "payoutId": 1, "transactionRefNumber": "HDFC2026080112345" }
```

Sets status to PROCESSED, stores the UTR number, records processedAt. Errors if already processed.

GET `/api/v1/analytics/admin/summary` — Admin dashboard — full platform summary

```
{
  "totalUsers": 1540, "totalProducts": 320, "totalOrders": 892,
  "totalRevenue": 8450000.00, // From PAID orders only
  "pendingOrders": 45, "completedOrders": 750 // DELIVERED status
}
```

10. Finance & Revenue APIs

Base path: `/api/v1/finance` — Admin console only, not called by storefront frontend

GET `/api/v1/finance/dashboard` — Admin finance dashboard

```
{
  "totalGrossRevenue": 1250000.00,    // Sum of net company profit from revenue logs
  "totalCommissionEarned": 125000.00, // Sum of commission_amount from revenue logs
  "pendingSellerPayouts": 85000.00,   // Sum of net_payable_amount for PENDING payouts
  "totalCompletedTransactions": 342    // Total payment records
}
```

GET `/api/v1/finance/revenue-logs` — Platform revenue ledger

```
[{
  "logId": 1, "orderId": 1,
  "grossAmount": 179998.00,    // Total order value
  "commissionAmount": 17999.80, // Platform 10% cut
  "gatewayFee": 539.99,        // Payment gateway fee
  "netCompanyProfit": 17459.81, // commission - gatewayFee
  "createdAt": "2026-08-02T12:10:00"
}]
```

11. Error Codes & Responses

Standard HTTP response codes returned by the API

FIELD	REQUIRED	NOTES
200 OK	—	Success — GET requests, updates, successful operations
201 Created	—	Resource created — POST /register, /products, /categories
400 Bad Request	—	Validation failed — missing required fields, invalid email format, quantity < 1
404 Not Found	—	Resource not found — invalid order/product/user ID; throws ResourceNotFoundException
500 Internal Error	—	Server error — duplicate email on register, empty cart on order, already-processed payout

Example Error Responses

400 VALIDATION ERROR

```
{ "status": 400, "errors": { "email": "Invalid email format", "name": "Name is required" } }
```

404 NOT FOUND

```
{ "status": 404, "message": "Order not found with id: 99" }
```

12. Database Schema (Reference)

13 tables managed via Flyway versioned migrations (V1–V8) — backend reference only

TABLE(S)	DESCRIPTION
users (V1)	Core user table for all roles: id, first_name, last_name, email (unique), password (nullable, BCrypt), google_id, profile_pic, phone_number, role, department, created_at
categories (V2)	id, name (unique), slug (auto-generated, unique), description, created_at
products (V2)	id, category_id → categories (SET NULL), seller_id → users (CASCADE), name, slug (unique), description, price, stock_quantity, image_url, is_active
carts / cart_items (V3)	carts: id, user_id (unique) → users, total_amount. cart_items: cart_id, product_id, quantity, sub_total
orders / order_items / addresses (V4)	orders: order_tracking_number, status, total_amount, payment_status. order_items: price snapshot at time of order. addresses: user saved addresses
payments / reviews (V5)	payments: transaction_id (unique), payment_method, amount, payment_status. reviews: one review per user per product, rating 1–5
otp_tokens, seller_profiles, platform_revenue_logs, payouts (V6–V8)	otp_tokens: 6-digit email OTP. seller_profiles: store_name, gst_number, pan_number, bank details. platform_revenue_logs: per-order ledger entry. payouts: seller_id, net_payable_amount, status

13. Complete API Summary

All 35 endpoints at a glance, in frontend-priority order

Authentication — /api/v1/auth

METHOD	ENDPOINT	DESCRIPTION	ACCESS
POST	/auth/register	Register new user	PUBLIC
POST	/auth/login	Password login	PUBLIC
POST	/auth/send-otp	Send email OTP	PUBLIC
POST	/auth/verify-otp	Verify OTP and login	PUBLIC
POST	/auth/google-login	Google OAuth login	PUBLIC

Categories — /api/categories

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	/categories	All categories	PUBLIC
GET	/categories/{id}	Category by ID	PUBLIC
POST	/categories	Create category	ADMIN
PUT	/categories/{id}	Update category	ADMIN
DELETE	/categories/{id}	Delete category	ADMIN

Products — /api/v1/products

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	/products	All products	PUBLIC
GET	/products/{id}	Product by ID	PUBLIC
GET	/products/slug/{slug}	Product by SEO slug	PUBLIC
GET	/products/category/{id}	Products by category	PUBLIC
GET	/products/seller/{id}	Products by seller	PUBLIC
POST	/products	Create product	SELLER
PUT	/products/{id}	Update product	SELLER
DELETE	/products/{id}	Delete product	SELLER

Cart — /api/v1/cart

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	/cart/user/{userId}	Get user cart	CUSTOMER
POST	/cart/user/{userId}/items	Add item to cart	CUSTOMER
PUT	/cart/user/{userId}/items/{itemId}	Update item qty	CUSTOMER
DELETE	/cart/user/{userId}/items/{itemId}	Remove item	CUSTOMER
DELETE	/cart/user/{userId}/clear	Clear entire cart	CUSTOMER

Orders — /api/v1/orders

METHOD	ENDPOINT	DESCRIPTION	ACCESS
POST	/orders	Place order from cart	CUSTOMER
GET	/orders/{id}	Order by ID	ALL
GET	/orders/user/{userId}	User order history	CUSTOMER

METHOD	ENDPOINT	DESCRIPTION	ACCESS
PUT	/orders/{id}/cancel	Cancel order	CUSTOMER
GET	/orders	All orders	ADMIN
PUT	/orders/{id}/status	Update status	ADMIN

Payments — /api/v1/payments

METHOD	ENDPOINT	DESCRIPTION	ACCESS
POST	/payments/process	Process payment	CUSTOMER
GET	/payments/order/{orderId}	Payment by order	ALL
POST	/payments/verify	Verify payment	ADMIN
POST	/payments/refund/{orderId}	Refund payment	ADMIN

Payouts & Analytics — /api/v1/payouts, /api/v1/analytics

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	/payouts/seller/{sellerId}	Seller payouts	SELLER
GET	/analytics/seller/{id}/summary	Seller dashboard	SELLER
GET	/payouts/pending	Pending payouts	ADMIN
POST	/payouts/process	Process payout	ADMIN
GET	/analytics/admin/summary	Admin dashboard	ADMIN

Finance — /api/v1/finance (Admin-only backend)

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	/finance/dashboard	Finance dashboard	ADMIN
GET	/finance/revenue-logs	Revenue ledger	ADMIN

E-Commerce Platform — Backend API Documentation

Spring Boot 3 · Java 17 · MySQL 8 · Flyway · Lombok · Spring Security 6

Version 1.0 · Base URL: <http://localhost:8080> · Reorganised for frontend integration, August 2026